University of Glasgow | School of Computing Science

Honours Individual Project Dissertation

Hypencoder for Biomedical Entity Linking

Steve Falkovsky

March 28th, 2026

Abstract

Biomedical literature and clinical records contain large amounts of information about diseases, drugs, and treatments, but this information is often expressed using ambiguous and inconsistent terminology. Without resolving these ambiguities, it becomes difficult to reliably interpret, aggregate, and retrieve biomedical knowledge from text, limiting the effectiveness of applications such as biomedical literature search and clinical decision support. To address this, natural language processing systems must perform entity linking, mapping mentions in text to standardised biomedical concepts.

Modern entity linking systems typically use efficient Bi-Encoder retrieval models to generate candidate entities in an initial retrieval stage. However, these models rely on fixed similarity functions between mention and entity representations, which may limit their ability to capture complex contextual relationships required for accurate biomedical disambiguation. Therefore, this dissertation investigates whether the Hypencoder architecture can improve first-stage biomedical entity linking by replacing the fixed similarity function used in standard Bi-Encoders with a more flexible neural network scoring model that adapts to each input mention.

The results show that context is useful only when kept local and limited, as short context windows improved retrieval more consistently than broader context, which often reduced performance. Increasing the depth of the Hypencoder scoring network did not improve performance, with the best results achieved by the shallowest configuration tested. The strongest Hypencoder model substantially outperformed the SapBERT baseline across all reported retrieval metrics, demonstrating significantly better ranking quality and showing that neural network scoring can capture relationships between mention context and candidate entities more effectively than conventional Bi-Encoder similarity.

Overall, the dissertation provides evidence that Hypencoder is a promising retrieval architecture for biomedical entity linking. At the same time, because efficiency was not evaluated directly, the work should be understood as establishing effectiveness rather than proving practical suitability for large-scale deployment.

Acknowledgements

Education Use Consent

Consent for educational reuse withheld. Do not distribute.

Contents

1 Introduction **1**

1.1 Motivation 1

1.2 The Problem 1

1.3 Proposed Solution 1

2 Background **2**

2.1 Biomedical Entity Linking Approaches 2

2.1.1 Alias Matching Entity Linking 2

2.1.2 Contextual Biomedical Entity Linking 3

2.1.3 Generative Entity Linking 4

2.2 Hypernetworks 5

2.3 Hypencoder 6

2.4 Research Questions 7

3 Methods **8**

3.1 Dataset Selection and Pre-processing 8

3.1.1 Input Truncation and Context Window Extraction 9

3.1.2 Context Window Configuration 9

3.1.3 Input Representation 10

3.2 Computational Environment and Constraints 10

3.3 Model Configurations 11

3.4 Training Strategy 11

3.4.1 Loss Function: In-Batch Negatives 12

3.4.2 Hyperparameters and Optimization 12

3.4.3 Training Procedure 12

3.5 Inference and Evaluation 14

3.5.1 Hypencoder Scoring Pipeline 14

3.5.2 Evaluation Metrics 15

4 Results **16**

4.1 Qualitative Error Analysis 16

4.1.1 Mismatched Abstraction Level 16

4.1.2 Insufficient Context 16

4.1.3 Semantically Similar Predictions 17

4.1.4 Implications for Interpreting the Results 17

4.2 RQ1 – How do different levels of context, specifically the mention passage and the entity definition, impact the accuracy of a Hypencoder? 18

4.3 RQ2 – How does the number of hidden layers in the Hypencoder scoring network affect retrieval accuracy? 19
4.4 RQ3 – How does the retrieval accuracy of a Hypencoder compare to that of a SapBERT Bi-Encoder? 20
4.4.1 Practical Implications 21
4.5 Result Summary 22

5 Conclusion 23
5.1 Summary 23
5.2 Reflection 24
5.3 Limitations and Future Work 24

Bibliography 26

1 Introduction

1.1 Motivation

Entity Linking (EL) is a core task in natural language processing that maps ambiguous mentions in text to canonical entities in a knowledge base, enabling downstream applications such as semantic search and question answering. In modern systems, EL is typically formulated as a two-stage Information Retrieval (IR) pipeline to balance efficiency and accuracy.

The first stage, candidate retrieval, uses Bi-Encoders to map mentions and entities into a shared dense vector space, enabling efficient large-scale retrieval through simple similarity functions such as dot product or cosine similarity. This allows millions of entities to be filtered in milliseconds.

The second stage, Reranking, employs a Cross-Encoder to perform a deep semantic comparison between the mention and the reduced candidate list. While highly accurate, the Cross-Encoder's computational overhead makes it impractical for use across an entire database.

1.2 The Problem

A critical bottleneck in the two-stage entity linking pipeline is the candidate retrieval stage. Bi-Encoders encode mentions and entities independently into fixed-length vectors and score them using a fixed similarity function such as the dot product or cosine similarity. While this makes retrieval efficient, it can limit the model's ability to capture the contextual distinctions required for difficult disambiguation. If the gold entity is not retrieved in the top-k candidates, the reranker cannot recover it, since it can only reorder the candidates it receives. As a result, retrieval errors propagate through the rest of the pipeline and constrain overall system performance.

1.3 Proposed Solution

Therefore, this project investigates the implementation and benchmarking of the Hypencoder, a recently proposed retrieval architecture designed to make first-stage candidate retrieval more expressive without losing the efficiency of dense retrieval methods. Instead of representing a mention using only a single fixed vector that is compared to entity vectors with a simple similarity function, the Hypencoder uses the mention to generate a lightweight neural network that takes in an entity vector and outputs a similarity score. This allows scoring to adapt to the input mention, rather than relying on the same fixed comparison for every query. In principle, this gives the retrieval stage more flexibility to model the kinds of contextual distinctions that standard Bi-Encoders often miss, while still remaining much cheaper than a Cross-Encoder.

The primary objective of this project is to evaluate whether the added flexibility of the Hypencoder can improve first-stage biomedical entity linking sufficiently to narrow the gap between efficient Bi-Encoders retrieval and more accurate but computationally expensive Cross-Encoder reranking. More broadly, it investigates the Hypencoder's usefulness as a stronger retrieval component in a two-stage architecture or, in some settings, as a sufficiently strong single-pass retriever, potentially offering a simpler and more computationally efficient solution for large-scale biomedical entity linking.

2 Background

Entity linking systems are commonly implemented as two-stage retrieval pipelines, where a first-stage retriever proposes a small set of candidate entities and a second-stage reranker refines their ordering (Figure 2.1). In biomedical settings, this pattern is especially common because ontologies are large and mentions are often ambiguous. In practice, the effectiveness of the full pipeline depends heavily on the quality of first-stage retrieval, which motivates research on more expressive retrieval architectures.

2.1 Biomedical Entity Linking Approaches

Biomedical Entity Linking presents unique challenges compared to general-domain tasks. Clinical text is often dense with synonyms (e.g., "heart attack" vs. "myocardial infarction") and highly ambiguous abbreviations.

The top performing biomedical entity linking systems have evolved along several different modelling paradigms. These were examined by Kartchner et al. (2023) and can broadly be grouped into three categories: (1) Alias matching models that focus on aligning synonymous entity names, (2) contextualized retrieval models that incorporate surrounding textual context to resolve ambiguous mentions, and (3) autoregressive generative models that formulate entity linking as a sequence generation task.

2.1.1 Alias Matching Entity Linking

One of the earliest approaches to biomedical entity linking relies on matching a mention in text to the names of entities stored in a biomedical knowledge base. These knowledge bases, such as UMLS or MeSH , contain large collections of biomedical concepts along with lists of alternative names or aliases that refer to the same concept, as detailed in Bodenreider (2004). In this setting, the task is to determine which concept in the knowledge base corresponds to a given mention extracted from a document.

Biomedical concepts often have many different textual variants. For example, the condition “myocardial infarction” may also appear in text as “heart attack”, while the drug “acetaminophen” may also be written as “paracetamol”. These different textual representations of the same concept are often referred to as *surface forms* or *aliases*. Entity linking systems based on alias matching attempt to resolve a mention by comparing its text directly with the names and aliases stored in the knowledge base.

Biomedical ontologies typically contain large synonym sets for each concept, which makes alias matching a practical starting point for entity linking. However, exact or approximate string matching remains limited when two mentions refer to the same concept using substantially different surface forms, or when lexically similar names correspond to different entities. This motivates approaches that learn dense representations of entity names, so that synonymous terms can be retrieved based on semantic similarity rather than only lexical overlap.

SapBERT by Liu et al. (2021) is a representative example of this paradigm. Built on top of PubMedBERT, SapBERT uses synonym relationships from UMLS to train an encoder in which

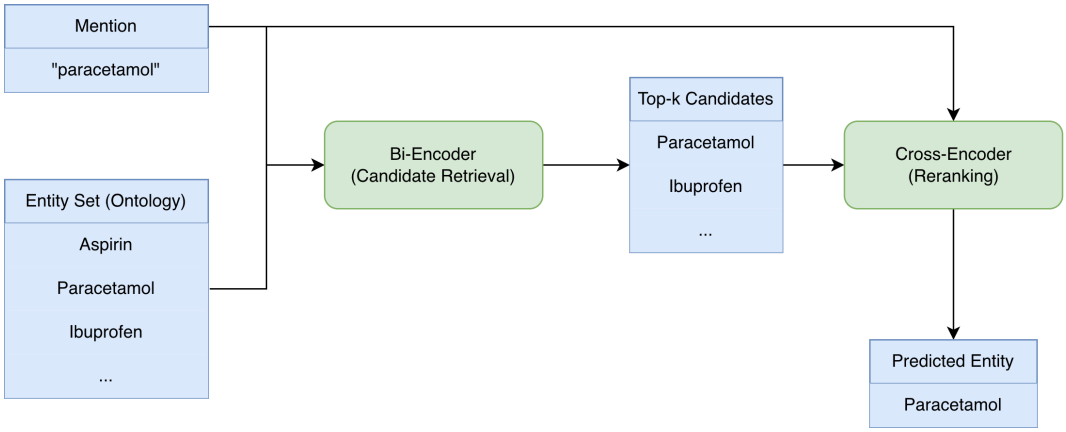

Figure 2.1: *Two-stage entity linking pipeline. A Bi-Encoder retrieves a small set of candidate entities from a large ontology, which are then reranked by a cross-encoder to select the most similar entity.*

different names for the same biomedical concept are mapped to similar vector representations. In this way, aliases and synonymous surface forms are clustered together in a shared embedding space, improving retrieval over large biomedical vocabularies. Although such models are more robust to variation in entity names than purely string-based methods, and have achieved strong performance on biomedical benchmarks, they still operate primarily on entity names and synonyms rather than the surrounding textual context.

As a result, they can struggle to resolve ambiguous mentions that require contextual reasoning. Zhang et al. (2022) argue that, for example, the abbreviation "ER" may refer to several biomedical concepts, including Emergency Room, Estrogen Receptor, or Endoplasmic Reticulum. Without incorporating contextual information from the surrounding sentence, models based purely on surface-form similarity cannot reliably resolve such ambiguity.

Another limitation of these models is that candidate entities are typically compared using simple similarity functions such as the dot product between embedding vectors. Killingback et al. (2025) claim that while this enables efficient large-scale retrieval, the resulting scoring function may lack the expressive capacity needed to capture complex semantic relationships between mentions and entities. These limitations motivate the development of contextual retrieval models that incorporate surrounding text when resolving ambiguous mentions, which are discussed in the following section.

2.1.2 Contextual Biomedical Entity Linking

To address the limitations of alias-based approaches, more recent biomedical entity linking systems incorporate contextual information from the surrounding text when resolving mentions. Rather than relying solely on surface-form similarity between a mention and entity names, these models encode the mention together with its textual context and compare it with representations of candidate entities in a shared embedding space. The central idea is that contextual cues in the sentence or document can disambiguate mentions whose surface forms alone are insufficient for accurate linking.

Early contextual biomedical entity linking systems did not abandon lexical-based retrieval entirely. Instead, many adopted hybrid architectures that combined contextual mention representations with lexical matching over ontology aliases. This reflected a practical limitation of biomedical EL: although ontologies such as UMLS contain millions of concepts, annotated datasets cover only a small fraction of them, so purely supervised contextual models often struggle to generalise

to unseen entities. MedLinker by Loureiro and Jorge (2020) illustrates this design. It represents mentions using contextual embeddings from a biomedical language model, but supplements these representations with approximate dictionary matching over entity aliases, using lexical similarity to retrieve candidates whose names resemble the mention text. In MedLinker, this string-based component acts as a fallback mechanism for zero-shot or sparsely observed entities, while the contextual component helps distinguish between candidates when local textual evidence is informative. More broadly, such hybrid systems show that context was recognised as important for disambiguation, but that lexical matching remained necessary because biomedical knowledge bases are extremely large and only weakly covered by labelled training data.

Subsequent work has focused not only on incorporating context into entity linking, but on learning stronger contextual mention representations under the severe supervision constraints of the biomedical domain. Because biomedical ontologies contain millions of entities while only a small fraction have labelled training mentions, an important challenge is how to train contextual encoders that can generalise beyond the limited set of manually annotated examples. KRISS Zhang et al. (2022) addresses this problem through knowledge-rich self-supervision. Rather than relying on gold entity linking annotations, it uses entity names from a biomedical ontology such as UMLS to identify matching occurrences in large collections of unlabeled biomedical text, thereby constructing contextual mention examples automatically. These automatically collected examples are used to train a contextual encoder to produce representations in which mentions of the same entity are more similar to each other than to mentions of different entities. At inference time, each entity is represented by multiple contextual examples, and a mention is linked to the entity whose representations are most similar. This is important because it shows that contextual biomedical entity linking is not only a matter of using context at inference time, but also of learning contextual representations at scale without depending entirely on scarce manual supervision.

Overall, contextual retrieval models represent a significant advancement over alias-based entity linking approaches. By incorporating information from the surrounding text, these models are better equipped to resolve ambiguous mentions and handle cases where lexical similarity alone is insufficient. However, this shift towards neural representations introduces new challenges. In contrast to traditional alias-based systems, which use highly efficient string matching over indexed entity names and aliases, many contextual models rely on dense vector similarity search over large entity sets, which can be computationally more demanding at scale. In addition, their performance often depends on the availability of suitable training signals, such as annotated mentions and entity descriptions. These limitations have motivated alternative approaches, which are discussed in the following section.

2.1.3 Generative Entity Linking

An alternative paradigm for entity linking formulates the task as a text generation problem rather than a retrieval task. Instead of retrieving candidate entities from a knowledge base and selecting the best match, a generative model directly produces the name or identifier of the target entity from the mention and its surrounding context.

In this setting, the model receives a sentence containing the mention and generates a textual representation of the corresponding entity. For example, given a sentence containing the mention "cold", the model may generate the canonical entity name *Common Cold*. Encoder-Decoder language models such as BART and T5 are commonly used for this purpose, as they have explicit cross-attention and can generate structured outputs based on the input context.

Generative approaches offer several advantages over traditional retrieval-based pipelines. Because the entire input is processed jointly, the model can use contextual information from the surrounding sentence when resolving ambiguous mentions. This allows the model to reason about the meaning of the mention in context rather than relying only on similarity between indepen-

dently encoded representations. In addition, generative models do not require storing vector embeddings for every entity in the knowledge base, which can reduce memory requirements.

Several recent studies, specifically by Yuan, Hongyi et al. (2022) and Yuan et al. (2022) have explored generative models for biomedical entity linking by adapting pretrained biomedical language models to generate entity names or identifiers directly. To ensure that generated outputs correspond to valid concepts in the ontology, these systems often apply constrained decoding. Under this approach, the model is restricted to generating only valid ontology names or identifiers rather than arbitrary text, substantially reducing hallucinations.

Despite these advantages, generative entity linking also introduces several practical challenges. Generative language models usually produce the output one token at a time. Because the entity name or identifier must be generated step by step, prediction is often slower than in retrieval-based systems, where the model only needs to compare the mention representation against precomputed entity representations. This can be problematic when linking entities across large biomedical corpora. Additionally, the model, as is, may produce outputs that do not correspond to valid entities in the knowledge base. Ensuring that generated outputs map exactly to ontology identifiers therefore often requires additional constraints or post-processing, which can be expensive.

For these reasons, retrieval-based methods remain the dominant approach for large-scale biomedical entity linking systems. Consequently, much recent research has focused on improving the effectiveness of retrieval architectures while maintaining their scalability. The Hypencoder architecture investigated in this work follows this line of research by introducing a more expressive retrieval model for scoring candidate entities.

2.2 Hypernetworks

To understand the Hypencoder, it is necessary to look at the history of Hypernetworks, first introduced by Ha et al. (2016). In Machine Learning, the behaviour of the model is determined by a set of parameters known as weights. During training, the model learns these weights from data so that it can map inputs to useful representations or predictions. Once training is complete, the learned weights remain fixed. This means that every input processed by the model is handled using the same set of parameters. In the case of a Bi-Encoder retrieval system, all mentions and entities are encoded using a single shared model with fixed weights. While this allows the system to process queries efficiently, it also means that the model cannot adjust representations based on the specific characteristics of an individual query.

A Hypernetwork changes this by using a fixed neural network to generates weights for another neural network based on a specific input. This process is known as dynamic weight generation. Prior work has shown that hypernetwork-based approaches are particularly effective in settings where a single fixed model must generalise across diverse inputs or tasks Chauhan et al. (2024). For example, in few-shot and continual learning, dynamically generated parameters allow models to adapt to new or evolving data distributions more effectively than static architectures.

Applying this idea to Entity Linking suggests a different approach to scoring candidate entities. Instead of relying on a fixed similarity function, such as a dot product between embeddings, the scoring process can adapt to the specific mention and its context. A recent line of work explores this idea in the context of information retrieval. In particular, Killingback et al. (2025) introduce the *Hypencoder*, a hypernetwork-based architecture that uses dynamic weight generation to construct a query-specific scoring network.

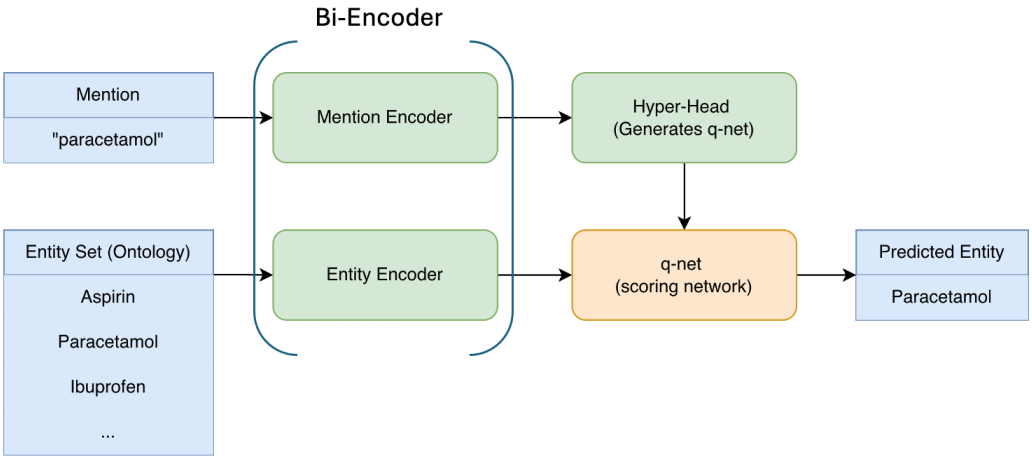

Figure 2.2: *Hypencoder retrieval architecture. The mention is encoded and passed to a hyper-head, which generates a query-specific scoring network (q-net). This generated network is applied to entity embeddings to score and rank candidate entities, producing the final prediction.*

2.3 Hypencoder

The Hypencoder Killingback et al. (2025) is a recently proposed architecture that introduces a method for improving dense retrieval by changing how similarity scores between queries and candidate entities are computed. In standard dense retrieval, queries and entities are encoded into vector representations and compared using a fixed similarity function, such as the dot product. In contrast, the Hypencoder replaces this fixed comparison with a query-conditioned scoring mechanism, as illustrated in Figure 2.2. In this setting, *query-conditioned scoring* means that the function used to score candidate entities is generated specifically for each input query, rather than being the same for all queries.

To achieve this, the Hypencoder uses a Hypernetwork to generate a custom, lightweight scoring network (q-net) on the fly from the query representation, which in this project corresponds to the mention and its surrounding context. The generated q-net is then used to score candidate entities. As a result, the scoring behaviour can adapt to the specific characteristics of the input query.

One key consequence of this design is that the scoring mechanism becomes context-dependent. In a traditional Bi-Encoder, the same similarity function is applied to every query. In contrast, the Hypencoder generates a scoring network tailored to the specific input, allowing the scoring behaviour to adapt depending on the mention being processed.

Another important property is that the scoring function becomes non-linear. Instead of computing similarity using a linear operation such as a dot product, the candidate entity representation is passed through a neural network whose parameters are generated for the current query. This potentially allows the model to capture more complex relationships between mentions and entities. Because the q-net is itself a neural network, its representational capacity can also be adjusted by modifying its architecture, for example by increasing the number of hidden layers. However, there is currently limited research on how changes in this architecture affect performance in specialised domains such as biomedical entity linking.

Finally, the Hypencoder retains one of the main advantages of Bi-Encoder retrieval: entity representations can still be pre-computed. Since the Hypernetwork only generates the scoring network for the query side, entity embeddings remain fixed and can be stored in advance. This

preserves the efficiency benefits of dense retrieval while introducing greater flexibility in how candidate entities are evaluated. As a result, the architecture remains suitable for large-scale retrieval tasks where millions of entities must be searched in real time.

2.4 Research Questions

The preceding review of the literature reveals a significant tension in Entity Linking: the speed of Bi-Encoders like SapBERT is necessary for scale, but their linear scoring method creates an accuracy bottleneck. While the Hypencoder architecture offers a dynamic, non-linear alternative, its application to the specialized biomedical domain requires rigorous evaluation.

To investigate the effectiveness of this approach, this project seeks to answer the following research questions:

- **RQ1: Contextual Impact.** How do different levels of context, specifically the mention passage and the entity definition, impact the accuracy of a Hypencoder?
- **RQ2: Model Complexity.** How does the number of hidden layers in the Hypencoder scoring network affect retrieval accuracy?
- **RQ3: Comparative Performance.** How does the retrieval accuracy of a Hypencoder compare to that of a SapBERT Bi-Encoder?

3 Methods

The primary goal of this research is to evaluate whether a single-stage Hypencoder can match or exceed the accuracy of a traditional Bi-Encoder in the biomedical domain. This chapter details the experimental framework, including the dataset selection, model architectures, and the training pipeline used to conduct this comparison.

3.1 Dataset Selection and Pre-processing

Several benchmark datasets exist for biomedical entity linking. In this work, the BC5CDR (BioCreative V Chemical–Disease Relation) corpus is used as the primary dataset Li et al. (2016). The dataset is grounded in MeSH (Medical Subject Headings), a biomedical vocabulary which is widely used for indexing biomedical literature. BC5CDR consists of 1,500 PubMed abstracts annotated with chemical and disease mentions, each linked to a corresponding MeSH concept. In total, it contains 15,935 annotated mentions and is divided into standard training, development, and test splits of 500 documents each. The presence of gold MeSH identifiers allows the dataset to be directly used for entity linking, where mentions are mapped to standardised concepts.

BC5CDR was selected for several reasons. First, the corpus provides high-quality expert annotations, which reduces label noise and ensures reliable evaluation. Second, it is widely used in biomedical NLP research, enabling direct comparison with existing methods. Finally, the dataset is relatively moderate in size, which makes it suitable for experimentation under limited computational resources. In this project, experiments were conducted using a single NVIDIA T4 GPU available through the Google Colab free tier, and the dataset size allows for multiple training iterations within practical runtime constraints while still producing statistically meaningful results.

A further design choice concerned the construction of the candidate entity set. Rather than retrieving from the full MeSH ontology at inference time, experiments were restricted to the subset of MeSH entities appearing in the annotated dataset split under consideration. In practice, this means that if the selected documents collectively contained 5,000 unique linked entities, only those 5,000 entities were included as candidates during scoring. This decision was made primarily for computational reasons. Full-ontology retrieval over MeSH would have required scoring each mention against a much larger candidate space, substantially increasing both memory usage and runtime. Given the limited computational resources available in this project, restricting the candidate set made repeated training and evaluation more practical.

This restricted-candidate setup has important implications for the interpretation of results. Most notably, it makes the linking task easier than a realistic deployment scenario in which the model must rank against the full MeSH ontology. The reported results should therefore be understood as measuring performance under a constrained candidate set, rather than full-scale ontology retrieval. In this setting, model performance reflects the ability to distinguish between the correct entity and a limited pool of observed ontology concepts, which is still useful for controlled comparison between architectures but may yield more optimistic scores than evaluation against the complete MeSH knowledge base.

Data Preparation: The raw data was processed into a structured format suitable for training, as seen in Table 3.1. Each entry in the final dataset consists of:

- **Mention:** The specific text string identified in the passage.
- **Context Passage (mention_text):** The surrounding text in the medical document.
- **Entity:** The entity name that the mention refers to.
- **Entity Definition:** A textual description of the entity retrieved from the MeSH ontology.
- **ID:** A MeSH ID used to link the mention to the right entity.
- **Aliases (not used in experiments):** All different aliases that an entity might have.

Table 3.1: *Example of a processed dataset entry used for biomedical entity linking experiments*

Field	Value
mention	Naloxone
mention_text	Naloxone reverses the antihypertensive effect of clonidine.
entity	Naloxone
aliases	Abello Brand of Naloxone Hydrochloride Boots Bristol Myers Squibb …
definition	A specific opiate antagonist that has no agonist activity. It is a competitive …
id	MESH:D009270

3.1.1 Input Truncation and Context Window Extraction

In practice, both the mention passage and the entity definition can be substantially longer than is efficient to process during training and inference. Longer inputs increase computational cost and memory usage, particularly because sequences within a batch must be padded to a common length. To maintain computational efficiency, input texts were truncated before being passed to the encoder.

Different truncation strategies were used for the two text fields. For entity definitions, the beginning of the definition was retained, as the opening portion typically contains the most informative description of the concept. For mention passages, simply retaining the first portion of the text would often exclude the local context surrounding the mention itself. Instead, a context-window extraction method was used: the mention string was located in the passage using regular expression matching, and a fixed-length window of surrounding characters was extracted. This approach preserves the immediate context most relevant to disambiguation while avoiding unrelated text elsewhere in the passage.

3.1.2 Context Window Configuration

A 128-character window was selected for training as a practical compromise. It was typically large enough to preserve the immediate semantic context of the mention, while remaining short enough to keep training computationally manageable.

Because the first research question investigates the role of contextual information in entity disambiguation, an additional no-context configuration was also included in the experimental design. In this setup, the same dataset was retained, but only the mention name and entity name were used. This is an important design choice because removing context introduces many duplicate mention-entity pairs: of the 5516 total entries, only 2656 are unique when context is discarded. As a result, this does not reflect the same level of difficulty as the contextual setting, but the comparison remains useful from an application perspective, since it reflects how each model variant would behave when applied directly to the available dataset.

3.1.3 Input Representation

The mention-side and entity-side inputs were constructed separately in order to follow the Bi-Encoder style architecture used in both the SapBERT baseline and the Hypencoder. In both cases, the name was concatenated with its associated contextual text using the special `[SEP]` token provided by the BERT tokenizer.

For the mention-side input, the mention string was combined with the surrounding passage context using the following format:

```
<mention name> [SEP] <mention passage context>
```

For example:

```
cold [SEP] The patient was treated for a cold after
experiencing several days of coughing and fever.
```

The entity-side input was constructed in a similar manner by combining the entity name with its textual definition:

```
<entity name> [SEP] <entity definition>
```

For example:

```
Common Cold [SEP] A viral infectious disease of the upper
respiratory tract that typically causes coughing, sore
throat, and nasal congestion.
```

This formatting explicitly separates the name from the broader context while still allowing the encoder to attend jointly to both components.

3.2 Computational Environment and Constraints

All experiments were conducted using the Google Colab environment. This platform was selected due to its accessibility and built-in GPU support, which enabled efficient experimentation. Its widespread availability also supports reproducibility by allowing other researchers to replicate the experimental setup with minimal infrastructure requirements.

The experimental setup used a single NVIDIA T4 GPU with 16 GB of VRAM. The implementation was based primarily on PyTorch and the Hugging Face Transformers library. This software stack was chosen because it is widely used in modern natural language processing research, well supported, and compatible with the pretrained models and training workflows required for this project.

The original Hypencoder implementation assumes access to multi-GPU cluster resources, which are not available in the present experimental setup. All experiments were conducted on a single NVIDIA T4 GPU, imposing constraints on batch size and overall memory usage. As a result, the codebase required substantial refactoring to operate within a memory-constrained, single-device environment, while preserving the core experimental methodology. Furthermore, experiments were restricted to configurations that could be executed on a single device, which reduced the scale of hyperparameter exploration.

The use of Google Colab also has implications for reproducibility. Although the platform increases accessibility, its software environment changes over time as package versions are updated automatically. To mitigate this, package versions were fixed where practical. However, complete long-term reproducibility cannot be guaranteed solely through the notebook environment, particularly for large dependencies that may be difficult or time-consuming to downgrade. For this reason, reproducibility is best supported through explicit dependency specification and documentation of the execution environment alongside the released code.

3.3 Model Configurations

Two primary model architectures were evaluated in this work: a SapBERT Bi-Encoder baseline and the proposed Hypencoder. Both models were initialized from the same pretrained biomedical encoder, SapBERT, in order to ensure that any observed performance differences were attributable to the scoring architecture rather than to differences in the underlying language model.

The first model serves as the baseline. SapBERT is a strong biomedical Bi-Encoder and is widely used for representation learning in biomedical entity linking. In the present setup, it was fine-tuned on the BC5CDR-derived linking data and used to encode mentions and candidate entities into a shared representation space. Candidate ranking was then performed using cosine similarity, providing a standard point of comparison for first-stage retrieval performance. This baseline was selected because it represents the conventional linear scoring paradigm that this project aims to test against.

However, standard SapBERT models are typically trained on entity names and synonyms in isolation, which limits their ability to utilise contextual information when disambiguating mentions in text. To address this limitation, this project fine-tunes SapBERT using inputs that include both the mention and its surrounding textual context, together with the corresponding entity and its definition. Incorporating contextual information is expected to improve disambiguation in cases where surface forms are ambiguous. Despite this, it remains unclear which types and amounts of context contribute most effectively to retrieval performance in biomedical entity linking. Consequently, this work systematically evaluates different contextual inputs and context lengths as part of the experimental design.

The second model is the Hypencoder, which replaces the fixed similarity scoring function with a query-conditioned scoring function. First, the SapBERT backbone produces contextualized embeddings for the input mention and its surrounding context. These embeddings are then passed to a hyperhead, which generates the weights of the q-net. The generated q-net scores each candidate entity through a non-linear function tailored to the current query.

To investigate RQ2 (Model Complexity), several Hypencoder variants were trained in which the depth of the generated q-net was varied while keeping the rest of the architecture fixed. Specifically, q-net configurations with 2, 4, 6, and 8 hidden layers were evaluated. This allowed the effect of scoring-network depth to be studied independently of the encoder backbone. The intention was to test whether increasing the representational capacity of the query-specific scoring function improved biomedical disambiguation performance.

Across both architectures, the comparison was kept as consistent as possible. The same dataset and preprocessing pipeline were used for all runs, with differences restricted to the scoring architecture itself. In this way, the baseline measures the performance of a strong biomedical Bi-Encoder, while the Hypencoder evaluates whether dynamic, non-linear scoring yields measurable gains on BC5CDR.

3.4 Training Strategy

The training procedure was designed to ensure a fair comparison between the SapBERT baseline and the Hypencoder model. Both were trained using the same data splits, batching procedure, optimiser settings, and training framework. All experiments were implemented using the HuggingFace Trainer API, providing a consistent and reproducible pipeline across both architectures.

Although the two models differ in how they compute mention–entity similarity, they were trained under the same ranking objective. This follows the general training formulation described

by Killingback et al. (2025), who noted that Hypencoder training is equivalent to Bi-Encoder training except that the inner-product similarity is replaced by the q-net scoring function.

3.4.1 Loss Function: In-Batch Negatives

Both models were trained using a cross-entropy objective with in-batch negatives. Under this training strategy, a mini-batch contains N gold mention–entity pairs:

$$(m_1, e_1), (m_2, e_2), \ldots, (m_N, e_N).$$

For each mention m_i, the model computes a relevance score against every entity in the batch, producing an $N \times N$ score matrix. The diagonal entry $s(m_i, e_i)$ corresponds to the correct mention–entity pair, while the off-diagonal entries $s(m_i, e_j)$ for $j \neq i$ are treated as negatives. The loss then encourages the score of the gold entity to be higher than the scores of all other entities in the same batch.

This formulation is computationally efficient because it reuses other gold entities in the batch as negative examples, avoiding the need to explicitly mine a separate set of negatives for each mention. It is also particularly well suited to biomedical entity linking in this setting, where the dataset is not fully shuffled and consecutive samples often originate from the same document. As a result, mentions within a batch are more likely to share similar context or belong to related concepts, making in-batch negatives more challenging. Training under these conditions encourages the model to learn fine-grained distinctions between closely related biomedical entities, rather than relying on coarse semantic similarity alone.

3.4.2 Hyperparameters and Optimization

As with the rest of the components, the same training hyperparameters and optimisation settings (Table 3.2) were used across all experimental runs unless a model-specific architectural difference required otherwise. Hyperparameters were selected through validation experiments, in which both training and validation loss were monitored to identify stable convergence behaviour and the point at which additional training yielded diminishing returns.

Optimisation used AdamW, which is the standard optimiser for fine-tuning transformer-based models and is also consistent with the optimisation setup used for the original Hypencoder training, as noted by Killingback et al. (2025). Mixed-precision (FP16) training was enabled to reduce GPU memory usage and improve throughput on the available NVIDIA T4 GPU.

A batch size of 64 was chosen as a compromise between hardware constraints and training effectiveness. This choice was important because the in-batch negative objective benefits from larger batch sizes: each additional mention-entity pair in the batch contributes extra negative examples for the other instances. The learning rate was set to 2×10^{-5}, which is a standard and generally stable choice for fine-tuning BERT-based encoders. A warmup ratio of 0.1 was used to reduce optimisation instability during the early stages of training.

Training was capped at 200 steps, based on preliminary experiments showing that both training and validation loss began to plateau beyond this point. This limit was adopted to avoid unnecessary computation while still allowing the models sufficient time to converge on the training data. Unless explicitly stated otherwise, all remaining optimiser and trainer settings followed the implementation defaults provided by the HuggingFace Transformers library.

3.4.3 Training Procedure

All experiments were initialised from the same pre-trained checkpoint, *cambridgeltl/SapBERT-from-PubMedBERT*. This model serves two roles in our study: as a standalone bi-encoder baseline,

Table 3.2: *Training hyperparameters used for all experiments*

Parameter	Value	Justification
Batch Size	64	Chosen as the largest stable batch size supported by the available hardware while also strengthening the in-batch negative training signal.
Optimizer	AdamW	Standard optimiser for transformer fine-tuning; provides stable optimisation for BERT-based encoders.
Learning Rate	2×10^{-5}	Standard fine-tuning learning rate for BERT-family models and found to produce stable convergence in preliminary experiments.
Warmup Ratio	0.1	Used to stabilise optimisation during the early training steps and reduce the risk of large parameter updates at the start of fine-tuning.
Max Steps	200	Selected based on observed plateauing of training and validation loss in preliminary runs.
Precision	FP16 (mixed precision)	Reduced memory usage and improved training throughput on NVIDIA T4 GPUs without changing the model architecture.

and as the transformer backbone within the Hypencoder architecture.

For the SapBERT baseline experiments, the model was fine-tuned directly. For the Hypencoder experiments, the SapBERT backbone was not frozen; instead, it was fine-tuned jointly with the additional Hypencoder components. This allowed the underlying encoder to adapt to the contextual biomedical entity linking task while keeping the optimisation setting consistent across experiments.

The mention-side and entity-side inputs were constructed separately. The mention-side input consisted of the target mention together with its surrounding passage context, while the entity-side input consisted of the entity name together with its definition. Relevance was then computed using the model-specific scoring function: cosine similarity for the SapBERT baseline and q-net scoring for Hypencoder.

Models were evaluated on the validation set every 10 training steps. Training was run for a maximum of 200 steps, as preliminary experiments showed that both training and validation loss decreased rapidly during the early stages of training before beginning to plateau. Across all experiments, the overall shape of the training and validation loss curves was highly consistent, with both losses stabilising near the end of training. As a result, the best-performing checkpoint typically occurred close to the final training step. For this reason, the final model used for each experiment was the checkpoint obtained at step 200. This approach ensured a consistent training procedure across all model variants while avoiding additional variability introduced by checkpoint selection. The same evaluation and stopping procedure was applied uniformly across all experiments.

Table 3.3: *Training and Validation Loss Milestones*

Milestone	Step	Training Loss	Validation Loss
Initial	10	2.0150	2.1199
Middle (Convergence start)	100	0.7868	0.9615
Convergence	200	0.6498	0.8877

Figure 3.1 and Table 3.3 summarize the observed loss trajectory. Training and validation loss decreased continuously throughout the interval. The largest improvements occurred during

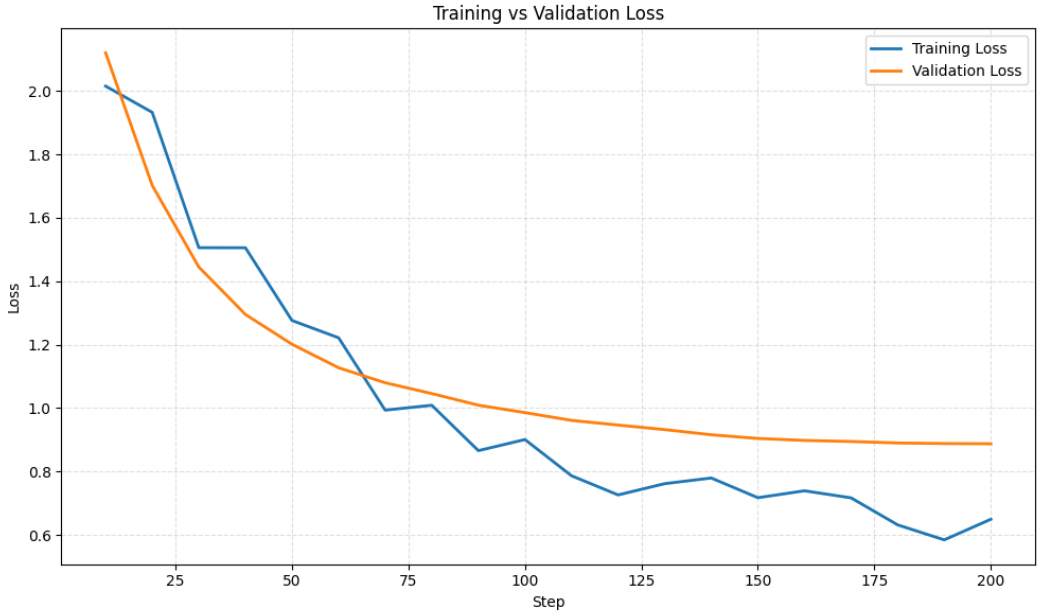

Figure 3.1: *Training and validation loss plot*

the earlier stages of training, with smaller gains in validation loss after approximately step 100, suggesting diminishing returns from further optimisation.

Several alternative hyperparameter configurations were also explored, but these did not produce substantial improvements in validation performance. Given the relatively small dataset and the diminishing returns observed beyond 200 steps, training was capped at 200 steps for all reported experiments.

3.5 Inference and Evaluation

Once the models were trained, a separate inference pipeline was developed to evaluate their performance on the BC5CDR test set. This section describes the mechanism for searching the entity ontology and the metrics used to answer the research questions.

3.5.1 Hypencoder Scoring Pipeline

Inference follows a fixed scoring pipeline that mirrors the training-time input construction and model architecture:

1. **Pre-processing:** Each test example is processed identically to the training data. The mention is trimmed to the selected context window, and structured inputs are constructed for both the mention and entity representations.
2. **Query encoding:** The mention and its context are encoded using the transformer backbone to produce a contextualised query representation.
3. **q-net generation:** The query representation is passed to the hypernetwork head, which generates the weights and biases of a query-specific scoring network (q-net).
4. **Entity scoring:** The generated q-net is applied to candidate entity representations to compute relevance scores, which are used for ranking.

For comparison, the SapBERT Bi-Encoder baseline follows a simpler pipeline. The mention and entity inputs are encoded independently using the same transformer backbone, and relevance is computed directly using cosine similarity between their embeddings.

3.5.2 Evaluation Metrics

Performance was assessed using Accuracy@k and Mean Reciprocal Rank (MRR). These metrics were chosen because biomedical entity linking can be formulated as a ranking problem: for each mention, the model must assign scores to a set of candidate entities and rank the correct entity as highly as possible.

Accuracy@k measures whether the correct entity appears within the top k candidates returned by the model. Formally, for a set of N evaluated mentions, Accuracy@k is defined as

$$\text{Accuracy@}k = \frac{1}{N}\sum_{i=1}^{N} 1\{\text{rank}_i \leq k\},$$

where rank_i is the rank position of the correct entity for mention i, and $1\{\cdot\}$ is an indicator function that takes the value 1 when the condition is true and 0 otherwise.

The following values of k were used:

- **Accuracy@1:** Measures how often the model ranks the correct entity first. This is the most important metric for evaluating the system as a standalone one-stage linker.
- **Accuracy@5 and Accuracy@10:** Measure whether the correct entity appears within the top 5 or top 10 ranked candidates. These metrics are useful for analysing near-misses and for assessing whether the correct entity remains highly ranked even when it is not placed first.

Mean Reciprocal Rank (MRR) provides a more position-sensitive evaluation of ranking quality. For each mention, if the correct entity appears at rank r_i, its reciprocal rank is given by $\frac{1}{r_i}$. The MRR over a set of N evaluated mentions is then:

$$\text{MRR} = \frac{1}{N}\sum_{i=1}^{N} \frac{1}{r_i}.$$

- **Sensitivity to Position:** Accuracy@k treats all positions within the top k equally. Whether the correct answer is at rank 1 or rank 10, the "Accuracy@10" remains the same. MRR rewards the system more for placing the correct answer at rank 1 (1.0) than at rank 2 (0.5).
- **Penalty for Lower Ranks:** MRR naturally incorporates a "decay". The difference between being at rank 1 and 2 is much larger (0.5) than the difference between rank 10 and 11 (≈ 0.009), which aligns with user behavior in search and recommendation contexts.

Together, Accuracy@k and MRR provide complementary perspectives on system performance. Accuracy@k captures whether the correct entity is retrieved within a practically useful cut-off, while MRR reflects the overall quality of the ranking by penalising lower-ranked correct predictions more strongly.

Because evaluation was performed on a restricted candidate set rather than the full MeSH ontology, these metrics should be interpreted as measuring comparative ranking performance under controlled conditions rather than full-scale ontology retrieval performance.

4 Results

4.1 Qualitative Error Analysis

Before presenting the quantitative results for each research question, it is useful to examine several recurring error patterns observed in the model's predictions. These qualitative observations provide important context for interpreting the evaluation metrics reported later in the chapter. In particular, because this task is evaluated using exact entity matching, some predictions counted as errors are genuinely incorrect, whereas others are semantically close to the gold-standard entity but differ in specificity or ontology level.

A manual inspection of incorrect predictions revealed three recurring patterns: mismatches in abstraction level, insufficient contextual information, and predictions that were semantically close to the correct entity but did not exactly match the annotated label.

4.1.1 Mismatched Abstraction Level

In some cases, the model fails to map a mention to the appropriate higher-level concept and instead predicts a more specific or locally related entity. This suggests that lexical overlap within the immediate context may sometimes influence the model more strongly than the broader conceptual meaning of the mention.

Table 4.1: *Example of an abstraction-level mismatch*

Field	Value
Mention	delusions
Correct entity	Cognition Disorders
Top prediction	Confusion
Context	progressive confusion, aggression, auditory hallucinations and delusions

In the example provided in Table 4.1, the model predicts *Confusion* rather than the broader concept *Cognition Disorders*. The surrounding sentence contains a closely related symptom term, which may have biased the prediction toward a nearby but overly specific concept. This type of error highlights the difficulty of linking surface mentions to the correct level of abstraction within a biomedical ontology.

4.1.2 Insufficient Context

A second source of error arises when the context window is too limited to support reliable disambiguation. In such cases, the mention itself may be vague, and the truncated surrounding text does not provide enough information for the model to infer the intended biomedical concept.

In the example shown in Table 4.2, the mention *subcellular degeneration* is highly underspecified when viewed in isolation. The short extracted context does not clearly indicate the precise

Table 4.2: Example of an error associated with limited context

Field	Value
Mention	subcellular degeneration
Entity	Nerve Degeneration
Top Match	Disease
Provided Context	ecovery" period. Electron microscopy of the myocardium revealed subcellular degeneration and marked mitochondrial changes after a single dose. Histopath

biomedical process being described, making it difficult for the model to recover the intended entity. More broadly, this suggests that entity linking performance may be sensitive to the amount and quality of contextual information available to the encoder.

4.1.3 Semantically Similar Predictions

A large proportion of apparent errors involve predictions that are semantically very close to the gold-standard entity. In these cases, the predicted concept is often a near-synonym, a closely related subtype, or a neighbouring concept in the ontology hierarchy.

Table 4.3: Examples of semantically similar predictions

Mention	Correct Entity	Predicted Entity
oculomotor dysfunction	Eye Diseases	Ocular Motility Disorders
amphetamine	Amphetamine	Amphetamines
anemia	Anemia	Anemia, Hemolytic, Autoimmune
diastolic dysfunction	Ventricular Dysfunction, Left	Ventricular Dysfunction

The examples in Table 4.3 highlight a limitation of strict exact-match evaluation in biomedical entity linking. Although the predicted entity is counted as incorrect, in several cases it remains clinically or semantically close to the target label. This is particularly common in biomedical ontologies such as MeSH, where related concepts may differ only by plurality, specificity, or taxonomic position.

4.1.4 Implications for Interpreting the Results

These error patterns are important for interpreting the quantitative results presented later in this chapter. Some errors reflect genuine failures of disambiguation, particularly where the model selects a concept at the wrong abstraction level or where the available context is insufficient. However, other errors arise because the evaluation requires an exact match even when the predicted concept is semantically close to the annotated label.

This does not invalidate the use of Accuracy@k or MRR, but it does indicate that exact-match performance alone may understate the practical usefulness of the system in some cases. The quantitative results should therefore be interpreted alongside the qualitative nature of the model's mistakes, especially when predictions differ from the gold label only in specificity or ontology granularity.

These observations also suggest that some errors might be reduced through lightweight post-processing, such as string normalization or rule-based handling of singular and plural variants.

However, such methods were not incorporated into the final system, as they would introduce additional heuristics and could also create false positive matches. The results reported in this chapter therefore reflect the behaviour of the core retrieval model without manual correction rules.

4.2 RQ1 – How do different levels of context, specifically the mention passage and the entity definition, impact the accuracy of a Hypencoder?

This research question investigates how different amounts of contextual information affect the performance of a one-stage Hypencoder for medical entity disambiguation. Specifically, it examines the contribution of two contextual fields available in the dataset: the mention passage and the entity definition. The central aim is to determine whether adding context improves retrieval accuracy, and whether performance is sensitive to a mismatch between the context length used during training and the context length used at inference time.

As discussed in Chapter 3, the underlying encoder was initialised from SapBERT, which was originally designed for biomedical entity representation based primarily on entity names rather than extended contextual passages. To enable its use in a contextual disambiguation setting, the encoder was fine-tuned on training examples that included contextual information using truncated mention passages and entity definitions. This makes RQ1 not only a question about whether context is useful, but also about how well a model derived from a name-focused encoder adapts to different context window sizes.

To study the effect of context length, one Hypencoder model was trained using examples with a 128-character context window. At inference time, this model was evaluated with different mention-passage context lengths ranging from 0 to 512 characters. Context window size was measured in characters rather than tokens, as this made manual inspection of truncation behaviour more straightforward during analysis, even though token length remains the quantity that determines computational cost internally.

For RQ1, evaluation is reported using Accuracy@1 and Accuracy@5 only. These metrics are sufficient here because the aim is to assess whether contextual information improves top-ranked retrieval performance, rather than to analyse finer-grained ranking differences across the candidate list. MRR is introduced later in RQ3, where the comparison between retrieval architectures makes overall ranking quality more central to the analysis.

Table 4.4: *Performance of the 2-layer Hypencoder trained with a 128-character context window and evaluated with varying inference-time context lengths*

Inference context window	Accuracy@1	Accuracy@5
Name only	0.8666	0.9500
64 characters	0.8698	0.9657
128 characters	0.8698	0.9648
256 characters	0.7886	0.9336
512 characters	0.6510	0.8504

The results in Table 4.4 show that introducing a small amount of local context leads to a modest improvement over the name-only baseline. The best Accuracy@5 score is achieved with a 64-character context window, while the best Accuracy@1 score is shared by the 64-character and 128-character settings. This indicates that short local context can be beneficial, but that the gain over name-only matching is limited.

This relatively small improvement is consistent with the nature of SapBERT, which was originally pretrained for terminology-focused biomedical representation and is therefore already strong at matching entities from surface forms alone. As a result, context appears to provide only incremental benefit when the mention string itself is already highly informative.

A more striking pattern is the sharp decline in performance as the context window becomes larger than the size used during training. Accuracy@1 falls from 0.8698 at 128 characters to 0.7886 at 256 characters and 0.6510 at 512 characters. A similar decline is observed for Accuracy@5. This suggests that additional context does not necessarily improve disambiguation, and that beyond a certain point, it may instead introduce distracting information that weakens the signal associated with the target mention.

One likely explanation is that longer mention passages often contain references to multiple biomedical entities or broader clinical themes, causing the representation of the local mention to become diluted by less relevant surrounding text. A second likely factor is the mismatch between training and inference conditions: because the model was trained using 128-character contexts, substantially longer inputs may fall outside the distribution it learned to encode effectively.

Overall, the results indicate that limited local context is beneficial, but only up to a relatively small window size. For this model, short context windows preserve useful disambiguating information, whereas larger windows introduce noise and reduce retrieval accuracy. In answer to RQ1, the inclusion of context can improve one-stage Hypencoder performance, but the effect is modest, and performance is highly sensitive to both context length and alignment between training-time and inference-time input format.

4.3 RQ2 – How does the number of hidden layers in the Hypencoder scoring network affect retrieval accuracy?

This research question investigates how the depth of the generated scoring network (q-net) affects the performance of the Hypencoder. In this experiment, only the architecture of the scoring network was varied, specifically the number of hidden layers. All other conditions were kept constant, including the pretrained encoder backbone (SapBERT), the training dataset, and the number of training steps.

The aim of this experiment was to determine whether increasing the capacity of the scoring network improves retrieval accuracy, or whether a simpler architecture is sufficient when the input representations are already produced by a strong pretrained biomedical encoder.

For RQ2, performance is reported using Accuracy@1. This metric is sufficient here because the goal is to compare which architecture most reliably places the correct entity at the top rank, rather than to analyse broader ranking behaviour across multiple candidate positions.

Table 4.5: *Performance of Hypencoder scoring networks with varying depth*

Hidden layers	Accuracy@1
2	**0.8700**
4	0.8562
6	0.8475
8	0.8199

The results in Table 4.5 show a clear monotonic decline in performance as the number of hidden layers increases. The 2-layer model achieved the highest Accuracy@1 of 0.8700, while progressively deeper architectures performed worse, with the 8-layer model dropping to 0.8199.

Under the training conditions used in this study, increasing model depth therefore did not improve Hypencoder accuracy.

One likely explanation is that the available training data were insufficient to support more complex scoring networks. Deeper architectures contain more parameters and typically require more examples to learn robust task-specific patterns without overfitting. With approximately 5,500 annotated training instances, the simpler architectures may have been better matched to the scale of the dataset.

A second explanation is that the scoring network operates on top of SapBERT representations, which are already highly informative for biomedical terminology. In this setting, the downstream network may only need to learn a relatively simple transformation from pretrained embeddings to relevance scores. Additional hidden layers may therefore add complexity without contributing useful representational capacity.

A further possibility is that deeper architectures require either more training steps or more careful hyperparameter tuning to realise their potential. Because all models were trained the same way, the deeper variants may not have been trained long enough to make effective use of their additional parameters.

Overall, the results indicate that, for this task and dataset, shallow scoring networks provide the best balance between capacity and generalisation. In answer to RQ2, increasing the depth of the generated scoring network does not improve performance; instead, the strongest results are achieved with the simplest architecture evaluated.

This finding differs from the trend reported in the original Hypencoder paper, where performance improved beyond two layers, peaked at around four layers, remained stable at six, and declined at eight layers Killingback et al. (2025). This suggests that the optimal depth of the generated scoring network is not universal, but depends on factors such as dataset size, task structure, and training conditions. In the present setting, the combination of dataset size limitations and a strong pre-trained encoder appears to favour a shallower architecture.

4.4 RQ3 – How does the retrieval accuracy of a Hypencoder compare to that of a SapBERT Bi-Encoder?

This research question compares the best-performing Hypencoder configuration against a traditional Bi-Encoder baseline on the same biomedical entity linking task. The aim is to determine whether the Hypencoder architecture provides a measurable retrieval advantage over SapBERT when both models are applied to the same dataset and evaluated under the same conditions.

For this comparison, the 2-layer Hypencoder was selected, as it achieved the best performance in RQ2. It was compared against the SapBERT Bi-Encoder baseline fine-tuned for the same task. For RQ3, evaluation is reported using Accuracy@1, Accuracy@5, Accuracy@10, and Mean Reciprocal Rank (MRR). While Accuracy@k shows whether the correct entity appears within the top-k retrieved results, MRR provides additional information about how highly each model ranks the correct entity overall.

The results, displayed in Table 4.6 and Figure 4.1, show a substantial and consistent advantage for the Hypencoder across all reported metrics. The largest practical difference is visible at Accuracy@1, where the Hypencoder achieves 0.8700 compared with 0.4420 for the Bi-Encoder. This indicates that the Hypencoder is far more likely to place the correct entity at the top rank, which is particularly important in a one-stage retrieval setting.

The advantage remains strong at larger values of k. Although the Bi-Encoder improves noticeably when evaluated at top-5 and top-10, it still remains well below the Hypencoder. This suggests

Table 4.6: Comparison of Hypencoder and Bi-Encoder performance

Metric	Hypencoder (2-layer, SapBERT base)	Bi-Encoder (SapBERT)
Accuracy@1	**0.8700**	0.4420
Accuracy@5	**0.9648**	0.7587
Accuracy@10	**0.9739**	0.8153
MRR	**0.9116**	0.5737

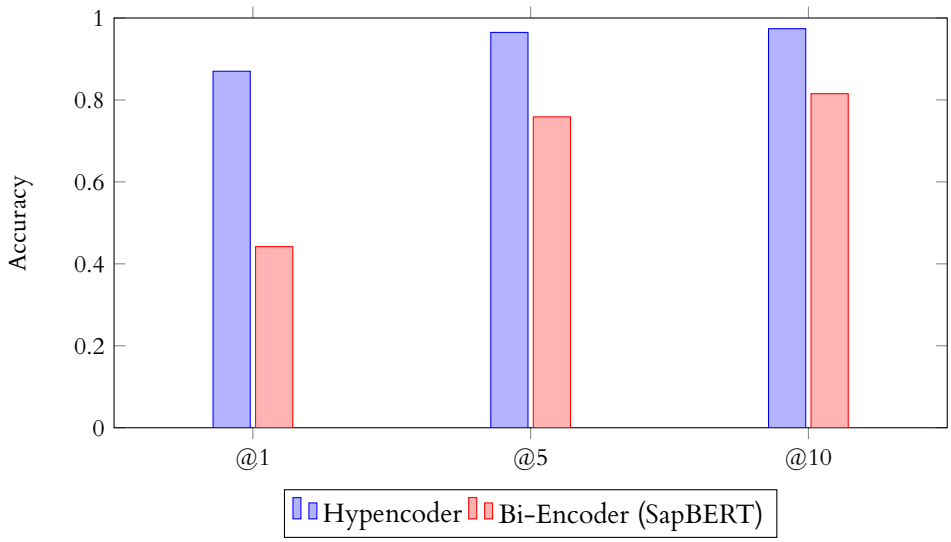

Figure 4.1: Comparison of Hypencoder and Bi-Encoder accuracy across top-k retrieval levels.

that the Bi-Encoder often retrieves the correct entity somewhere within the broader candidate set, but is much less effective at ranking it near the top.

The MRR results reinforce this pattern. The Hypencoder achieves an MRR of 0.9116, substantially higher than the Bi-Encoder's 0.5737. This indicates that, on average, the Hypencoder ranks the correct entity much earlier in the candidate list. In contrast, the Bi-Encoder frequently places the correct entity lower down the ranking, even when it is retrieved within the top 10.

One plausible explanation for this difference is architectural. Standard Bi-Encoders rely on separately encoded representations combined using relatively simple similarity functions, such as inner product or cosine similarity. The Hypencoder framework was proposed precisely to address this limitation by replacing fixed linear similarity with a more expressive query-conditioned scoring network. The present results are consistent with that argument: for this entity linking task, the Hypencoder appears better able to capture the more complex relationships between mention context and entity representation that are needed for accurate ranking.

In answer to RQ3, the Hypencoder substantially outperforms the traditional SapBERT Bi-Encoder on this biomedical entity linking task. It achieves higher exact-match retrieval accuracy and much stronger overall ranking quality, indicating that the query-conditioned scoring mechanism provides a meaningful advantage over conventional Bi-Encoder similarity.

4.4.1 Practical Implications

From an effectiveness perspective, these results suggest that the Hypencoder is a credible candidate for replacing a more traditional two-stage retrieval setup with a single-stage model for this task.

In particular, its high Accuracy@5 indicates that the correct entity is usually ranked within a very small candidate set, while its strong Accuracy@1 shows that it frequently succeeds without requiring a second reranking stage.

However, this conclusion should be interpreted with some caution. This study evaluated retrieval quality only, and did not measure training cost, indexing cost, or inference latency. These factors are essential when assessing whether a Hypencoder-based system is preferable in practice. A full system comparison would therefore require additional experiments under a common hardware setup, measuring the efficiency of training, encoding, and retrieval for Bi-Encoder, Hypencoder, and cross-encoder style approaches.

4.5 Result Summary

Overall, the results presented in this chapter show that Hypencoder is highly effective for the biomedical entity linking setting considered in this dissertation. Short local context was found to be beneficial, while larger context windows reduced performance. Simpler scoring networks outperformed deeper architectures, suggesting that additional model complexity was unnecessary under the available training conditions. Most importantly, the best-performing Hypencoder substantially outperformed the context-finetuned SapBERT Bi-Encoder baseline across all reported retrieval metrics. These findings provide a consistent answer to the research questions and form the basis for the broader interpretation given in the next chapter.

5 Conclusion

5.1 Summary

This dissertation investigated whether the Hypencoder architecture can improve first-stage biomedical entity linking by replacing the fixed similarity function of a traditional Bi-Encoder with a query-conditioned scoring network. The work focused on the BC5CDR dataset and used SapBERT as the underlying biomedical encoder and as the main baseline for comparison. Beyond reproducing the general idea of Hypencoder, the project adapted the architecture to a biomedical entity linking setting, implemented a practical experimental pipeline for training and evaluation in a constrained single-GPU environment, and released the resulting models and datasets publicly to support reproducibility and further experimentation.

The dissertation addressed three main questions. First, it examined the effect of contextual information on Hypencoder performance. The results showed that context can be helpful, but only in a limited and carefully controlled form: short local context improved retrieval more consistently than larger context windows, which often introduced noise. Second, it examined the effect of q-net depth. Here, increasing the complexity of the generated scoring network did not improve performance; instead, the strongest results were obtained with the shallowest configuration evaluated. Third, it compared the best-performing Hypencoder against a fine-tuned SapBERT Bi-Encoder baseline on the same task. On BC5CDR, the Hypencoder achieved a substantial improvement in retrieval effectiveness, particularly in top-ranked accuracy, indicating that q-net scoring can offer a meaningful advantage over conventional Bi-Encoder similarity for biomedical entity linking.

Taken together, these findings show that Hypencoder is a promising architecture for improving retrieval quality in this domain. The results provide evidence that the model can outperform a strong SapBERT-based first-stage baseline under a controlled evaluation setup, while also showing that its performance depends on design choices such as context representation and scoring-network depth. In that sense, the project achieved its central research aim at the level of retrieval effectiveness: it demonstrated that a Hypencoder-based approach can substantially strengthen first-stage ranking quality on the biomedical entity linking task studied here.

However, the broader practical objective stated in the introduction was more demanding than accuracy alone. The motivation for Hypencoder is not simply to outperform a standard Bi-Encoder, but to do so while preserving the low-latency characteristics that make Bi-Encoders suitable for first-stage retrieval. That question remains unresolved in this dissertation. Although entity representations remain pre-computable, Hypencoder still requires the generation of a query-specific scoring network and the application of that network to candidate representations at inference time. This introduces additional computational work that may be negligible, acceptable, or prohibitive depending on the retrieval setting, hardware, index size, and implementation details. Since this study did not measure inference latency, throughput, memory usage, or indexing cost under a common hardware setup, it cannot yet conclude that Hypencoder is a practically superior first-stage system.

5.2 Reflection

Several aspects of the project went well. Most importantly, the dissertation succeeded in turning a recent research idea into a functioning biomedical entity linking system that could be trained and evaluated in a repeatable way. Multiple Hypencoder variants were implemented and compared under controlled settings, which made it possible to investigate specific research questions rather than relying on the original paper and its limited array of models. The experimental process also benefited from careful logging and manual inspection of retrieved candidates. This made it easier to identify unusual model behaviour, spot failure cases, and interpret why some configurations performed better than others. In practice, this combination of quantitative evaluation and qualitative inspection was one of the most useful parts of the project, because it helped move the work beyond simply reporting metrics.

Another positive outcome was that the project remained feasible despite substantial practical constraints. The work was carried out in a limited compute environment, and this required scaling the experiments carefully and making deliberate choices about what could realistically be tested. Although this restricted the size of the experimental setup, it also forced the project to stay focused on the most important comparisons. In retrospect, this improved the clarity of the dissertation, because the final evaluation remained closely tied to the main research questions rather than becoming too broad or unfocused.

However, not all parts of the project went smoothly. A significant amount of time was spent dealing with engineering and environment issues rather than directly investigating the research questions. In particular, working in Google Colab introduced instability because package versions could change over time, causing previously working code to break. This was especially problematic for a project built on a research codebase with several dependencies. As a result, some effort was diverted away from experimentation and into maintenance, debugging, and compatibility fixes. Although this was frustrating, it also highlighted an important lesson about machine learning research in practice: reproducible experimentation depends not only on model design, but also on controlling the software environment in which the work is conducted.

A further difficulty was the decision to build on top of the original Hypencoder codebase without critically reassessing its suitability early enough. At the start of the project, this seemed like the most efficient choice, particularly because the code came from the original paper. In practice, however, the codebase was difficult to adapt and extend for the needs of this dissertation. Small fixes gradually accumulated into technical debt, and tasks that should have been straightforward, such as adding customised evaluation routines or running modified experiments, became unnecessarily time-consuming. Looking back, this was probably the most costly technical decision in the project.

If the project were to be done again, the main change would be to prioritise software structure much earlier. Rather than continuing to work around limitations in the existing codebase, it would have been better either to refactor core components early or to reimplement a smaller but cleaner version of the system specifically for the dissertation. That would likely have reduced debugging time, improved reproducibility, and made later experimentation easier.

Overall, the project showed that research progress in machine learning depends not only on having a good model idea, but also on making sound decisions about experimentation, tooling, and reproducibility. In that sense, the difficulties encountered were not separate from the research process, but part of learning how to carry it out effectively.

5.3 Limitations and Future Work

A key limitation of this study is that all models were trained and evaluated on BC5CDR. Although this allows controlled comparison between architectures and training settings, it limits the extent

to which the results can be assumed to generalise to other biomedical entity linking datasets. BC5CDR is relatively small, which further constrains the strength of broader claims about the robustness of the Hypencoder approach. The findings in this dissertation should therefore be interpreted as evidence that Hypencoder performs strongly on BC5CDR under the evaluated setup, rather than as proof that the same advantage will necessarily hold across biomedical entity linking more generally.

A second limitation concerns the baseline comparison. Although the SapBERT Bi-Encoder baseline was fine-tuned under the same project pipeline and evaluated on the same task, SapBERT was originally developed primarily for biomedical concept representation and synonym matching rather than for context-heavy entity linking . As a result, the comparison does not fully establish how Hypencoder would perform against models that are specifically pretrained or architecturally designed to make stronger use of contextual information during retrieval. A useful direction for future work would therefore be to compare Hypencoder not only against standard SapBERT-style Bi-Encoders, but also against retrieval models that are explicitly pretrained or adapted for contextual entity disambiguation.

A third limitation is that retrieval efficiency was not evaluated. Although the results show that Hypencoder improves retrieval effectiveness on BC5CDR, this dissertation does not establish whether those gains are compatible with the latency and resource constraints of a practical first-stage system. Future work should therefore measure inference latency, throughput, memory usage, and indexing cost under a common hardware setup, so that effectiveness can be assessed alongside deployment feasibility.

Beyond these limitations, a natural next step would be to repeat the experiments on a larger and more diverse benchmark such as MedMentions Mohan and Li (2019). This would make it possible to test whether the same trends hold under a broader ontology and more varied mention contexts. In addition, MedMentions provides semantic group annotations (e.g., drugs, diseases, genes), enabling more fine-grained analysis of model performance across different biomedical categories. This would allow investigation into whether Hypencoder is consistently effective or whether its performance varies across entity types. Cross-dataset evaluation would provide much stronger evidence of generalisability, while comparisons against stronger context-aware baselines and explicit efficiency measurements would provide a more complete assessment of whether Hypencoder is a practically superior first-stage biomedical entity linking model.

Overall, this dissertation has shown that Hypencoder is a promising approach for biomedical entity linking and that it can substantially improve first-stage retrieval effectiveness over the evaluated SapBERT baseline on BC5CDR. Although further work is needed to establish its generalisability and practical efficiency at scale, the results provide clear evidence that query-conditioned scoring is a worthwhile direction for future biomedical retrieval research.

Bibliography

Bodenreider, O. (2004), 'The Unified Medical Language System (UMLS): integrating biomedical terminology', *Nucleic Acids Research* **32**(90001), 267D–270.
URL: *https://academic.oup.com/nar/article-lookup/doi/10.1093/nar/gkh061*

Chauhan, V. K., Zhou, J., Lu, P., Molaei, S. and Clifton, D. A. (2024), 'A brief review of hypernetworks in deep learning', *Artificial Intelligence Review* **57**(9), 250.
URL: *https://link.springer.com/10.1007/s10462-024-10862-8*

Ha, D., Dai, A. and Le, Q. V. (2016), 'HyperNetworks'. arXiv:1609.09106 [cs].
URL: *http://arxiv.org/abs/1609.09106*

Kartchner, D., Deng, J., Lohiya, S., Kopparthi, T., Bathala, P., Domingo-Fernández, D. and Mitchell, C. (2023), A Comprehensive Evaluation of Biomedical Entity Linking Models, *in* 'Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing', Association for Computational Linguistics, Singapore, pp. 14462–14478.
URL: *https://aclanthology.org/2023.emnlp-main.893*

Killingback, J., Zeng, H. and Zamani, H. (2025), 'Hypencoder: Hypernetworks for Information Retrieval'. arXiv:2502.05364 [cs].
URL: *http://arxiv.org/abs/2502.05364*

Li, J., Sun, Y., Johnson, R. J., Sciaky, D., Wei, C.-H., Leaman, R., Davis, A. P., Mattingly, C. J., Wiegers, T. C. and Lu, Z. (2016), 'BioCreative V CDR task corpus: a resource for chemical disease relation extraction', *Database* **2016**, baw068.
URL: *https://academic.oup.com/database/article-lookup/doi/10.1093/database/baw068*

Liu, F., Shareghi, E., Meng, Z., Basaldella, M. and Collier, N. (2021), 'Self-Alignment Pretraining for Biomedical Entity Representations'. arXiv:2010.11784 [cs].
URL: *http://arxiv.org/abs/2010.11784*

Loureiro, D. and Jorge, A. M. (2020), MedLinker: Medical Entity Linking with Neural Representations and Dictionary Matching, *in* J. M. Jose, E. Yilmaz, J. Magalhães, P. Castells, N. Ferro, M. J. Silva and F. Martins, eds, 'Advances in Information Retrieval', Vol. 12036, Springer International Publishing, Cham, pp. 230–237. Series Title: Lecture Notes in Computer Science.
URL: *https://link.springer.com/10.1007/978-3-030-45442-5$_2$9*

Mohan, S. and Li, D. (2019), 'MedMentions: A Large Biomedical Corpus Annotated with UMLS Concepts'. arXiv:1902.09476 [cs].
URL: *http://arxiv.org/abs/1902.09476*

Yuan, H., Yuan, Z. and Yu, S. (2022), Generative Biomedical Entity Linking via Knowledge Base-Guided Pre-training and Synonyms-Aware Fine-tuning, *in* 'Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies', Association for Computational Linguistics, Seattle, United States, pp. 4038–4048.
URL: *https://aclanthology.org/2022.naacl-main.296*

Yuan, Hongyi, Yuan, Zheng, Gan, Ruyi, Zhang, Jiaxing, Xie, Yutao and Yu, Sheng (2022), 'BioBART: Pretraining and Evaluation of A Biomedical Generative Language Model'. arXiv:2204.03905 [cs].
URL: *http://arxiv.org/abs/2204.03905*

Zhang, S., Cheng, H., Vashishth, S., Wong, C., Xiao, J., Liu, X., Naumann, T., Gao, J. and Poon, H. (2022), Knowledge-Rich Self-Supervision for Biomedical Entity Linking, *in* 'Findings of the Association for Computational Linguistics: EMNLP 2022', Association for Computational Linguistics, Abu Dhabi, United Arab Emirates, pp. 868–880.
URL: *https://aclanthology.org/2022.findings-emnlp.61*